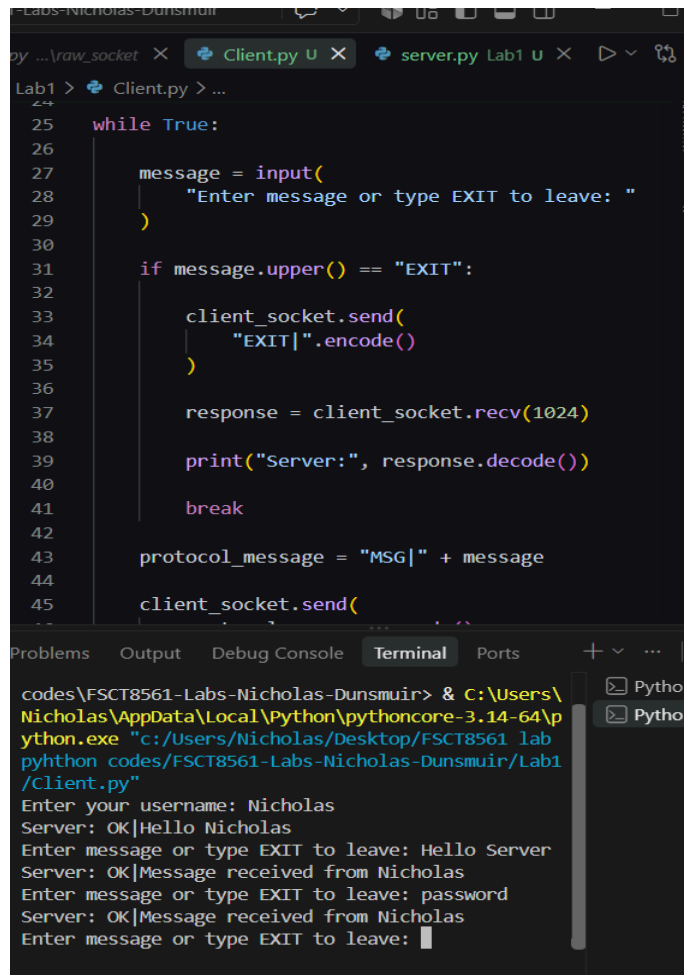


Lab1 Nicholas Dunsmuir

Lab1 GitHub link

<https://github.com/NDunsmuir-BCIT/FSCT8561-Labs-Nicholas-Dunsmuir/tree/main/Lab1>

PART 5:



```
Lab1 > Client.py > ...
25 while True:
26     message = input(
27         "Enter message or type EXIT to leave: "
28     )
29
30
31     if message.upper() == "EXIT":
32
33         client_socket.send(
34             "EXIT".encode()
35         )
36
37         response = client_socket.recv(1024)
38
39         print("Server:", response.decode())
40
41         break
42
43     protocol_message = "MSG|" + message
44
45     client_socket.send(
46         protocol_message.encode()
47     )
48
49     # Receive data from the server
50     response = server_socket.recv(1024)
51
52     print("Client:", response.decode())
53
54     # Send data to the server
55     client_socket.send(
56         response.encode()
57     )
58
59     # Close the client socket
60     client_socket.close()
61
62     # Close the server socket
63     server_socket.close()
64
65     # Exit the program
66     break
```

codes\FSCT8561-Labs-Nicholas-Dunsmuir> & C:\Users\Nicholas\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/Nicholas/Desktop/FSCT8561 lab python codes/FSCT8561-Labs-Nicholas-Dunsmuir/Lab1 /Client.py"

Enter your username: Nicholas
Server: OK|Hello Nicholas
Enter message or type EXIT to leave: Hello Server
Server: OK|Message received from Nicholas
Enter message or type EXIT to leave: password
Server: OK|Message received from Nicholas
Enter message or type EXIT to leave:

Part 6:

The screenshot shows a VS Code editor with a file named `server.py` open. The code is a Python script for a simple socket server. It listens for connections, receives a message, and responds based on the content. The terminal window at the bottom shows the execution of the script, displaying the server's state and the received messages from a client.

```
Lab1 > server.py > ...
66         client_socket.send(
67             "ERROR|Message cannot be empty"
68         )
69
70     elif len(content) > 200:
71         client_socket.send(
72             "ERROR|Message too long".encode()
73         )
74
75     else:
76         print(username + " says:", content)
77
78         client_socket.send(
79             ("OK|Message received from " + username).encode()
80         )
81
82     elif command == "EXIT":
83
84         client_socket.send(
85             "OK|Goodbye".encode()
86         )
87
```

Terminal Output:

```
python.exe "c:/Users/Nicholas/Desktop/FSCT8561 lab
python codes/FSCT8561-Labs-Nicholas-Dunsmuir/Lab1
/server.py"
Server is waiting for a connection...
Connected by: ('127.0.0.1', 60174)
Received: HELLO|Nicholas
Username: Nicholas
Received: MSG|Hello Server
Nicholas says: Hello Server
Received: MSG|password
Nicholas says: password
```

Part 8:

1.

The screenshot shows a terminal window where a Python client script is being executed. The user enters their username, and the server responds with an error message because the username is required. The user then enters a message, and the server responds with a confirmation message.

```
c:\Users\Nicholas\Desktop\FSCT8561 lab
python codes/FSCT8561-Labs-Nicholas-Dunsmuir/Lab1
/Client.py
Enter your username:
Server: ERROR|Username required
Enter message or type EXIT to leave: 
```



```
Problems Output Debug Console Terminal Ports
PS C:\Users\Nicholas\Desktop\F SCT8561 lab pyhthon
codes\F SCT8561-Labs-Nicholas-Dunsmuir> & C:\Users\
Nicholas\AppData\Local\Python\pythoncore-3.14-64\p
ython.exe "c:/Users/Nicholas/Desktop/F SCT8561 lab
pyhthon codes/F SCT8561-Labs-Nicholas-Dunsmuir/Lab1
/Client.py"
Enter your username: nick
Server: ERROR|Unknown command
Server: OK|Hello nick
Enter message or type EXIT to leave: [ ]
```

6.

Part 10 Questions:

1. The clients `recv()` or `send()` will show a connection error, this will most likely cause the client program to crash.
2. Using python's threading that allows the server to spawn separate thread tasks for each client connection
3. TCP is a continuous byte stream and not message based. The transport layer breaks the data into packets for transport and depending on network conditions the packets can arrive over several `recv()` calls.
4. TCP is only to ensure the package arrives in order at the transport layer and doesn't provide any encryption or verification to the data.
5. This can introduce username spoofing or command injection as well as many others
6. A client could send oversized messages to the server hindering other connections or causing a crash due to lack of available memory,
7. I'd add password tokens to require users to log in before having full access to be able to input commands to the server
8. By implementing TLS to the sockets to encrypt the data in transit.
9. Prevent empty messages and usernames and checking message length restrictions
10. Message rate limiting to prevent DoS attacks, proper logging and monitoring of the server connections and session tokens.

Challenge Section:

Bob sees Alices message :

```

1  import socket
2  import threading
3  import sys
4
5  HOST = "127.0.0.1"
6  PORT = 12345
7
8  def listen_for_messages(sock):
9      """Background thread to print incoming broadcast mess
10     while True:
11         try:
12             data = sock.recv(1024)
13             if not data:
14                 break
15             print(f"\n{data.decode()}\nEnter message or t
16         except:
17             break
18
19     client_socket = socket.socket(socket.AF_INET, socket.SOCK
20     client_socket.connect((HOST, PORT))
21
22     username = input("Enter your username: ")

```

Problems Output Debug Console Terminal Ports + v ... | [] X

```

PS C:\Users\Nicholas\Desktop\FSC8561 lab pyhton
codes\FSC8561-Labs-Nicholas-Dunsmuir\lab1> cd .\C
hallenge\
PS C:\Users\Nicholas\Desktop\FSC8561 lab pyhton
codes\FSC8561-Labs-Nicholas-Dunsmuir\lab1\Challen
ge> python .\Client.py
Enter your username: Bob
Server: OK|Hello Bob
Enter message or type EXIT to leave:
Alice: hello Bob its Alice
Enter message or type EXIT to leave: 

```

Python: ser...

Python: Cli...

pyth... [] [X]

Alice Sent message to the server:

```

1  import socket
2  import threading
3  import sys
4
5  HOST = "127.0.0.1"
6  PORT = 12345
7
8  def listen_for_messages(sock):
9      """Background thread to print incoming broadcast
10     while True:
11         try:
12             data = sock.recv(1024)
13             if not data:
14                 break
15             print(f"\n{data.decode()}\nEnter message: ")
16         except:
17             break
18
19  client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
20  client_socket.connect((HOST, PORT))
21
22  username = input("Enter your username: ")

```

Problems Output Debug Console Terminal Ports + v

```

codes\F SCT8561-Labs-Nicholas-Dunsmuir> & C:\Users\
Nicholas\AppData\Local\Python\pythoncore-3.14-64\p
ython.exe "c:/Users/Nicholas/Desktop/FSCT8561 lab
python codes/FSCT8561-Labs-Nicholas-Dunsmuir/Lab1
/Challenge/Client.py"
Enter your username: Alice
Server: OK|Hello Alice
Enter message or type EXIT to leave: hello Bob its
Alice
Enter message or type EXIT to leave:
OK|Message sent
Enter message or type EXIT to leave: 

```

Server showed both connected at the same time and server saw message from alice:

Lab1 > Challenge > Client.py > ...

```
1  import socket
2  import threading
3  import sys
4
5  HOST = "127.0.0.1"
6  PORT = 12345
7
8  def listen_for_messages(sock):
9      """Background thread to print incoming broadcast mess
10     while True:
11         try:
12             data = sock.recv(1024)
13             if not data:
14                 break
15             print(f"\n{data.decode()}\nEnter message or ")
16         except:
17             break
18
19  client_socket = socket.socket(socket.AF_INET, socket.SOCK
20  client_socket.connect((HOST, PORT))
21
22  username = input("Enter your username: ")
```

Problems Output Debug Console Terminal Ports

[DISCONNECT ERROR] Client ('127.0.0.1', 54544) error: [WinError 10054] An existing connection was forcibly closed by the remote host
[CLOSED] Connection with ('127.0.0.1', 54544) closed.
[NEW CONNECTION] Connected by ('127.0.0.1', 65327)
[NEW CONNECTION] Connected by ('127.0.0.1', 50253)
[REGISTERED] ('127.0.0.1', 50253) set username to 'Bob'
[REGISTERED] ('127.0.0.1', 65327) set username to 'Alice'
[CHAT] Alice: hello Bob its Alice

Pyth...  

Python: Cli...

python

Challenge Reflection Questions:

1. The original code for the server only had a single thread running, so it blocked on `accept()` waiting for a connection and also blocked on `recv()` waiting for data. This stopped it from handling new requests.
2. Threading allows for the server to delegate each client connection as its own independent variable. Keeping the main server loop open to listen for other connections and not block them.
3. When one disconnects the server only stops the thread that is attached to that specific client, leaving the others unaffected.
4. There could be data leaked between client sessions, broadcast spoofing and DoS attacks
5. Storing the same two or more usernames on a server can overwrite session records and change the routing of incoming messages
6. Enable the server to check for duplicate usernames and reject ones already used. We can also add session tokens tied to each connection socket

Security Analysis:

Changing from a single connection setup on the server, to a multi connection and persistent application greatly increases the servers attack surface. It can also create a vulnerability where the user can try to send commands out of order. Without proper message framing the messages can become altered when using `recv(1024)`. Keeping TCP connections open in this scenario can expose the server to attacks like spoofing and DoS and malicious connections can remain connected indefinitely, since TCP doesn't provide any application-layer confidentiality, authorization or payload validation.

Clients can send unknown command keywords to force the server to terminate unexpectedly, which could disconnect all clients. With all of the data being unencrypted, the usernames and passwords would be clear text for MitM attackers to grab them. There is also a lack of data integrity and validation from messages sent and received, unwanted data could be transferred easily.

A few options to fix these issues would be to implement an encryption to the transferred data to ensure it isn't clear text. Adding a hard limit to the size of the messages to hinder memory bloat of the server. Checking the username entry requests have input and there isn't a copy of one already on the server, as well as, adding passwords and signed token to validate the username accounts.